

The 42 Philosophers Project: A Beginner-Friendly Guide

1. Project Goal and the Dining Philosophers Problem

1.1. Project Goal

The primary goal of the 42 Philosophers project is to introduce you to the fundamental concepts of **multithreading**, **synchronization**, and **resource management** in a concurrent programming environment. You will learn how to create and manage multiple threads of execution, and more importantly, how to prevent common issues like **race conditions** and **deadlocks** using synchronization primitives such as **mutexes**.

This project is a practical exercise in understanding how concurrent processes interact and share resources, which is a critical skill in modern software development.

1.2. The Dining Philosophers Problem

The Dining Philosophers Problem is a classic problem in computer science used to illustrate synchronization issues and solutions in concurrent programming. Imagine a round table with a bowl of spaghetti in the center. Around the table are several philosophers, and between each pair of philosophers is a single fork.

Here's the catch:

- **Eating:** To eat the spaghetti, a philosopher needs *two* forks: one from their left and one from their right.
- **Thinking/Sleeping:** When not eating, philosophers are either thinking or sleeping. They put down their forks when they finish eating and pick them up again when they want to eat.
- **Starvation:** The simulation stops if any philosopher dies of starvation. A philosopher dies if they don't eat within a certain `time_to_die` period.

The challenge is to design a program that allows all philosophers to eat without any of them starving, while also avoiding **deadlocks** (where no philosopher can eat because they are all waiting for a fork that another philosopher holds) and **race conditions** (where the outcome depends on the unpredictable timing of multiple threads accessing shared resources).

Your program will simulate this scenario, ensuring that philosophers can pick up forks, eat, sleep, and think, all while adhering to strict timing and synchronization rules. The simulation must continue until a philosopher dies or, optionally, until all philosophers have eaten a specified number of times.

2. Required Functions and Their Roles

This section details the essential functions you'll need to implement or utilize to build your Philosophers project. Understanding the purpose of each function is crucial for a successful implementation.

2.1. Core Threading Functions

`pthread_create`

Purpose: This function is used to create a new thread of execution within the calling process. Each philosopher in your simulation will be represented by a separate thread.

Synopsis:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *
(*start_routine) (void *), void *arg);
```

- `thread`: A pointer to a `pthread_t` variable, which will store the ID of the newly created thread.
- `attr`: A pointer to a `pthread_attr_t` structure that specifies thread attributes (e.g., detach state, stack size). For this project, you can usually pass `NULL` for default attributes.
- `start_routine`: A pointer to the function that the new thread will execute. This function must take a `void *` argument and return a `void *`.

- `arg` : A pointer to an argument that will be passed to the `start_routine` function. This is how you can pass data to your philosopher threads.

Example Snippet:

```
#include <pthread.h>
#include <stdio.h>

void *philosopher_routine(void *arg)
{
    int philo_id = *(int *)arg;
    printf("Philosopher %d is alive!\n", philo_id);
    // ... philosopher's logic (eating, sleeping, thinking)
    return NULL;
}

int main()
{
    pthread_t philo_thread;
    int philo_id = 1;

    if (pthread_create(&philo_thread, NULL, philosopher_routine, &philo_id) !=
0)
    {
        perror("Failed to create thread");
        return 1;
    }
    // ... main thread continues
    pthread_join(philo_thread, NULL); // Wait for the philosopher thread to
finish
    return 0;
}
```

`pthread_join`

Purpose: This function waits for the termination of a specific thread. In your project, the main thread might use `pthread_join` to wait for all philosopher threads to complete their execution (though in the Philosophers project, the simulation often stops based on a death condition or eating count, not necessarily all threads joining).

Synopsis:

```
int pthread_join(pthread_t thread, void **retval);
```

- `thread` : The ID of the thread to wait for.
- `retval` : A pointer to a `void *` that will store the return value of the terminated thread.

Example Snippet: (See `pthread_create` example above for usage)

`pthread_detach`

Purpose: This function detaches a thread, meaning that its resources will be automatically reclaimed by the system when it terminates, without requiring another thread to call `pthread_join`. This is often used for threads that don't need to be explicitly waited upon by the main thread.

Synopsis:

```
int pthread_detach(pthread_t thread);
```

- `thread` : The ID of the thread to detach.

Example Snippet:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void *detached_philosopher_routine(void *arg)
{
    int philo_id = *(int *)arg;
    printf("Philosopher %d is running (detached)!\n", philo_id);
    sleep(1); // Simulate some work
    printf("Philosopher %d is done.\n", philo_id);
    return NULL;
}

int main()
{
    pthread_t philo_thread;
    int philo_id = 1;

    if (pthread_create(&philo_thread, NULL, detached_philosopher_routine,
    &philo_id) != 0)
    {
        perror("Failed to create thread");
        return 1;
    }
    pthread_detach(philo_thread); // Detach the thread
    printf("Main thread continues without waiting for philosopher.\n");
    sleep(2); // Give detached thread time to finish
    return 0;
}
```

2.2. Mutex Functions for Synchronization

Mutexes (Mutual Exclusion) are fundamental for protecting shared resources from simultaneous access by multiple threads, thereby preventing race conditions. In the Philosophers project, each fork will be protected by a mutex.

`pthread_mutex_init`

Purpose: Initializes a mutex. You must initialize a mutex before using it.

Synopsis:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *attr);
```

- `mutex`: A pointer to the `pthread_mutex_t` variable to be initialized.
- `attr`: A pointer to a `pthread_mutexattr_t` structure that specifies mutex attributes. For this project, you can usually pass `NULL` for default attributes.

Example Snippet:

```
#include <pthread.h>

pthread_mutex_t fork_mutex[5]; // Array of mutexes for 5 forks

void init_fork_mutexes(int num_philosophers)
{
    for (int i = 0; i < num_philosophers; i++)
    {
        if (pthread_mutex_init(&fork_mutex[i], NULL) != 0)
        {
            perror("Failed to initialize mutex");
            // Handle error
        }
    }
}
```

`pthread_mutex_destroy`

Purpose: Destroys a mutex, releasing any resources it might hold. You should destroy a mutex when it's no longer needed, typically at the end of your program, to prevent resource leaks.

Synopsis:

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- `mutex`: A pointer to the `pthread_mutex_t` variable to be destroyed.

Example Snippet:

```
// ... after all threads have finished and mutexes are no longer needed
void destroy_fork_mutexes(int num_philosophers)
{
    for (int i = 0; i < num_philosophers; i++)
    {
        if (pthread_mutex_destroy(&fork_mutex[i]) != 0)
        {
            perror("Failed to destroy mutex");
            // Handle error
        }
    }
}
```

pthread_mutex_lock

Purpose: Locks a mutex. If the mutex is already locked by another thread, the calling thread blocks until the mutex becomes available. This ensures that only one thread can access the protected resource at a time.

Synopsis:

```
int pthread_mutex_lock(pthread_mutex_t *mutex);
```

- `mutex`: A pointer to the `pthread_mutex_t` variable to lock.

Example Snippet:

```
// Inside a philosopher's routine when trying to pick up a fork
pthread_mutex_lock(&fork_mutex[left_fork_id]);
// Now the left fork is secured
```

pthread_mutex_unlock

Purpose: Unlocks a mutex. This releases the lock, allowing other threads to acquire it. It's crucial to unlock a mutex after you're done accessing the protected resource.

Synopsis:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- `mutex` : A pointer to the `pthread_mutex_t` variable to unlock.

Example Snippet:

```
// Inside a philosopher's routine after finishing eating and putting down forks  
pthread_mutex_unlock(&fork_mutex[left_fork_id]);  
pthread_mutex_unlock(&fork_mutex[right_fork_id]);  
// Forks are now available for others
```

2.3. Utility Functions

`usleep`

Purpose: Suspends the execution of the calling thread for a specified number of microseconds. This is useful for simulating the time philosophers spend eating, sleeping, or thinking.

Synopsis:

```
int usleep(useconds_t usec);
```

- `usec` : The number of microseconds to sleep.

Example Snippet:

```
#include <unistd.h>  
  
// Simulate eating for 200 milliseconds  
usleep(200 * 1000); // 200 milliseconds = 200,000 microseconds
```

`gettimeofday`

Purpose: Gets the current time, typically used for precise timing and calculating timestamps for logging philosopher states.

Synopsis:

```
int gettimeofday(struct timeval *tv, struct timezone *tz);
```

- `tv`: A pointer to a `struct timeval` where the current time will be stored. This structure contains `tv_sec` (seconds) and `tv_usec` (microseconds).
- `tz`: A pointer to a `struct timezone`. For most purposes, you can pass `NULL`.

Example Snippet:

```
#include <sys/time.h>
#include <stdio.h>

long long get_current_timestamp_ms()
{
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return (tv.tv_sec * 1000) + (tv.tv_usec / 1000);
}

// Usage:
// long long timestamp = get_current_timestamp_ms();
// printf("%lld Philosopher %d is eating\n", timestamp, philo_id);
```

printf

Purpose: Formats and prints output to the standard output. You'll use this extensively for logging the state changes of your philosophers as required by the subject.

Synopsis:

```
int printf(const char *format, ...);
```

Example Snippet: (See `gettimeofday` example above for usage)

memset, malloc, free, write

These are standard C library functions that you will use for memory management and basic I/O. `memset` for initializing memory, `malloc` for dynamic memory allocation (e.g., for philosopher structures), `free` for releasing allocated memory, and `write` for low-level output (though `printf` is generally preferred for formatted output).

3. Project Template Skeleton

Structuring your project correctly from the beginning is vital for readability, maintainability, and passing the 42 Norminette. This section provides a basic skeleton

for your `philo/` directory, including `main.c`, `philo.h`, and `utils.c`.

3.1. Directory Structure

Your mandatory part project should reside in a directory named `philo/`:

```
philo/  
├── Makefile  
├── main.c  
├── philo.h  
└── utils.c
```

3.2. `philo.h` - Header File

This file will contain all necessary includes, structure definitions, and function prototypes. It's crucial for organizing your data and ensuring proper communication between your source files.

```

#ifdef PHILO_H
#define PHILO_H

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/time.h>

// Define states for logging
#define STATE_EATING "is eating"
#define STATE_SLEEPING "is sleeping"
#define STATE_THINKING "is thinking"
#define STATE_FORK "has taken a fork"
#define STATE_DIED "died"

// Structure to hold simulation parameters and shared data
typedef struct s_data
{
    int num_philosophers;
    long long time_to_die;
    long long time_to_eat;
    long long time_to_sleep;
    int num_times_to_eat; // Optional argument
    long long start_time;
    int simulation_should_stop;
    pthread_mutex_t *forks; // Array of mutexes for forks
    pthread_mutex_t print_mutex; // Mutex for safe printing
    struct s_philo *philosophers; // Array of philosopher structures
} t_data;

// Structure to hold individual philosopher data
typedef struct s_philo
{
    int id;
    int left_fork_id;
    int right_fork_id;
    long long last_meal_time;
    int meals_eaten;
    pthread_t thread;
    t_data *data; // Pointer to shared simulation data
} t_philo;

// Function prototypes
// Initialization
int init_data(t_data *data, char **argv, int argc);
int init_philosophers(t_data *data);

// Simulation flow
void *philosopher_routine(void *arg);
void start_simulation(t_data *data);
void monitor_simulation(t_data *data);

// Actions
void philo_eat(t_philo *philo);
void philo_sleep(t_philo *philo);
void philo_think(t_philo *philo);
void take_forks(t_philo *philo);
void put_down_forks(t_philo *philo);

// Utilities

```

```
long long    get_time_ms(void);
void         print_status(t_philo *philo, char *status);
void         ft_usleep(long long milliseconds);
int          ft_atoi(const char *str);

// Cleanup
void         cleanup(t_data *data);

#endif
```

3.3. main.c - Main Program Logic

This file will handle argument parsing, data initialization, thread creation, and the main simulation loop (or monitoring thread).

```

#include "philo.h"

// Main function to parse arguments and start the simulation
int main(int argc, char **argv)
{
    t_data data;

    if (argc < 5 || argc > 6)
    {
        printf("Usage: ./philo number_of_philosophers time_to_die time_to_eat\n");
        return (1);
    }

    if (init_data(&data, argv, argc) != 0)
        return (1);
    if (init_philosophers(&data) != 0)
    {
        cleanup(&data);
        return (1);
    }

    start_simulation(&data);
    monitor_simulation(&data);

    cleanup(&data);
    return (0);
}

// Placeholder for start_simulation - creates philosopher threads
void start_simulation(t_data *data)
{
    int i;

    data->start_time = get_time_ms();
    i = -1;
    while (++i < data->num_philosophers)
    {
        data->philosophers[i].last_meal_time = data->start_time;
        if (pthread_create(&data->philosophers[i].thread, NULL,
                          philosopher_routine, &data->philosophers[i]) != 0)
        {
            // Handle error: set simulation_should_stop and join created
            data->simulation_should_stop = 1;
            printf("Error creating philosopher thread %d\n", i + 1);
            return ;
        }
    }
}

// Placeholder for monitor_simulation - checks for death or meal count
void monitor_simulation(t_data *data)
{
    int i;
    int all_eaten_enough;

    while (!data->simulation_should_stop)
    {
        i = -1;
        all_eaten_enough = 1;
    }
}

```

```

        while (++i < data->num_philosophers)
        {
            // Check if philosopher has died
            if (get_time_ms() - data->philosophers[i].last_meal_time > data->time_to_die)
            {
                print_status(&data->philosophers[i], STATE_DIED);
                data->simulation_should_stop = 1;
                break;
            }
            // Check if all philosophers have eaten enough (if optional arg is present)
            if (data->num_times_to_eat != -1 && data->philosophers[i].meals_eaten < data->num_times_to_eat)
                all_eaten_enough = 0;
        }
        if (data->num_times_to_eat != -1 && all_eaten_enough)
        {
            data->simulation_should_stop = 1;
            break;
        }
        ft_usleep(1); // Small delay to prevent busy-waiting
    }

    // Join all philosopher threads after simulation stops
    i = -1;
    while (++i < data->num_philosophers)
        pthread_join(data->philosophers[i].thread, NULL);
}

// Placeholder for philosopher_routine - the main loop for each philosopher thread
void *philosopher_routine(void *arg)
{
    t_philo *philo;

    philo = (t_philo *)arg;
    // Philosophers with even IDs start thinking to avoid immediate deadlock
    if (philo->id % 2 == 0)
        ft_usleep(philo->data->time_to_eat / 2); // Small delay

    while (!philo->data->simulation_should_stop)
    {
        philo_think(philo);
        take_forks(philo);
        philo_eat(philo);
        put_down_forks(philo);
        philo_sleep(philo);
    }
    return (NULL);
}

// Placeholder for init_data - parses arguments and initializes shared data
int init_data(t_data *data, char **argv, int argc)
{
    data->num_philosophers = ft_atoi(argv[1]);
    data->time_to_die = ft_atoi(argv[2]);
    data->time_to_eat = ft_atoi(argv[3]);
    data->time_to_sleep = ft_atoi(argv[4]);
    data->num_times_to_eat = -1; // Default to -1 (no limit)
    if (argc == 6)
        data->num_times_to_eat = ft_atoi(argv[5]);
}

```

```

    if (data->num_philosophers <= 0 || data->time_to_die < 0 ||
        data->time_to_eat < 0 || data->time_to_sleep < 0 ||
        (argc == 6 && data->num_times_to_eat <= 0))
    {
        printf("Error: Invalid arguments.\n");
        return (1);
    }

    data->simulation_should_stop = 0;
    data->forks = (pthread_mutex_t *)malloc(sizeof(pthread_mutex_t) * data-
>num_philosophers);
    if (!data->forks)
        return (1);
    data->philosophers = (t_philo *)malloc(sizeof(t_philo) * data-
>num_philosophers);
    if (!data->philosophers)
    {
        free(data->forks);
        return (1);
    }

    int i = -1;
    while (++i < data->num_philosophers)
    {
        if (pthread_mutex_init(&data->forks[i], NULL) != 0)
        {
            // Cleanup already initialized mutexes and allocated memory
            while (--i >= 0)
                pthread_mutex_destroy(&data->forks[i]);
            free(data->forks);
            free(data->philosophers);
            return (1);
        }
    }
    if (pthread_mutex_init(&data->print_mutex, NULL) != 0)
    {
        // Cleanup all mutexes and allocated memory
        while (--i >= 0)
            pthread_mutex_destroy(&data->forks[i]);
        free(data->forks);
        free(data->philosophers);
        return (1);
    }
    return (0);
}

// Placeholder for init_philosophers - initializes individual philosopher data
int init_philosophers(t_data *data)
{
    int i;

    i = -1;
    while (++i < data->num_philosophers)
    {
        data->philosophers[i].id = i + 1;
        data->philosophers[i].left_fork_id = i;
        data->philosophers[i].right_fork_id = (i + 1) % data->num_philosophers;
        data->philosophers[i].last_meal_time = 0; // Will be set at simulation
start
        data->philosophers[i].meals_eaten = 0;
        data->philosophers[i].data = data; // Link to shared data
    }
}

```

```

    }
    return (0);
}

// Placeholder for cleanup - frees allocated memory and destroys mutexes
void cleanup(t_data *data)
{
    int i;

    i = -1;
    while (++i < data->num_philosophers)
        pthread_mutex_destroy(&data->forks[i]);
    pthread_mutex_destroy(&data->print_mutex);
    free(data->forks);
    free(data->philosophers);
}

// Placeholder for philo_eat
void philo_eat(t_philo *philo)
{
    print_status(philo, STATE_EATING);
    pthread_mutex_lock(&philo->data->print_mutex);
    philo->last_meal_time = get_time_ms();
    philo->meals_eaten++;
    pthread_mutex_unlock(&philo->data->print_mutex);
    ft_usleep(philo->data->time_to_eat);
}

// Placeholder for philo_sleep
void philo_sleep(t_philo *philo)
{
    print_status(philo, STATE_SLEEPING);
    ft_usleep(philo->data->time_to_sleep);
}

// Placeholder for philo_think
void philo_think(t_philo *philo)
{
    print_status(philo, STATE_THINKING);
}

// Placeholder for take_forks
void take_forks(t_philo *philo)
{
    // Philosopher takes left fork first, then right fork
    // This order is crucial for preventing deadlocks in some strategies
    pthread_mutex_lock(&philo->data->forks[philo->left_fork_id]);
    print_status(philo, STATE_FORK);
    pthread_mutex_lock(&philo->data->forks[philo->right_fork_id]);
    print_status(philo, STATE_FORK);
}

// Placeholder for put_down_forks
void put_down_forks(t_philo *philo)
{
    pthread_mutex_unlock(&philo->data->forks[philo->left_fork_id]);
    pthread_mutex_unlock(&philo->data->forks[philo->right_fork_id]);
}

```

3.4. `utils.c` - Utility Functions

This file will contain helper functions that are not directly part of the core simulation logic but are essential for its operation, such as time handling and safe printing.

```

#include "philos.h"

// Returns current time in milliseconds
long long get_time_ms(void)
{
    struct timeval tv;

    gettimeofday(&tv, NULL);
    return ((tv.tv_sec * 1000) + (tv.tv_usec / 1000));
}

// Prints philosopher status with timestamp, protected by a mutex
void print_status(t_philo *philos, char *status)
{
    long long timestamp;

    pthread_mutex_lock(&philos->data->print_mutex);
    if (!philos->data->simulation_should_stop)
    {
        timestamp = get_time_ms() - philos->data->start_time;
        printf("%lld %d %s\n", timestamp, philos->id, status);
    }
    pthread_mutex_unlock(&philos->data->print_mutex);
}

// Custom usleep to handle smaller delays and check for simulation stop
void ft_usleep(long long milliseconds)
{
    long long start;

    start = get_time_ms();
    while ((get_time_ms() - start) < milliseconds)
        usleep(500); // Sleep for 0.5ms to reduce CPU usage
}

// Simple atoi implementation (you might use your libft's ft_atoi)
int ft_atoi(const char *str)
{
    int res = 0;
    int sign = 1;
    int i = 0;

    while (str[i] == ' ' || (str[i] >= '\t' && str[i] <= '\r'))
        i++;
    if (str[i] == '-')
    {
        sign = -1;
        i++;
    }
    else if (str[i] == '+')
        i++;
    while (str[i] >= '0' && str[i] <= '9')
    {
        res = res * 10 + (str[i] - '0');
        i++;
    }
    return (res * sign);
}

```

3.5. Makefile

Your `Makefile` must adhere to 42 standards, including `NAME`, `all`, `clean`, `fclean`, and `re` rules. It should compile with `-Wall -Wextra -Werror`.

```
NAME = philo

SRCS = main.c \
      utils.c

OBJS = $(SRCS:.c=.o)

CC = cc
CFLAGS = -Wall -Wextra -Werror -pthread

all: $(NAME)

$(NAME): $(OBJS)
    $(CC) $(OBJS) $(CFLAGS) -o $(NAME)

clean:
    rm -f $(OBJS)

fclean:
    rm -f $(OBJS) $(NAME)

re: fclean all

.PHONY: all clean fclean re
```

4. Understanding Concurrency Issues: Mutexes, Race Conditions, and Deadlocks

In multithreaded programming, where multiple threads execute concurrently and often share resources, certain problems can arise if not handled carefully. This section explains three critical concepts: race conditions, deadlocks, and how mutexes help prevent them.

4.1. Race Conditions

A **race condition** occurs when multiple threads try to access and modify the same shared resource concurrently, and the final outcome depends on the unpredictable order in which the threads execute. It's like a 'race' to see which thread gets to the resource first or in what order they perform their operations.

Analogy: Imagine two people, Alice and Bob, trying to update a single bank account balance simultaneously. The current balance is \$100.

- Alice wants to deposit 50. *She reads the balance(100), adds 50, and is about to write 150 back.*
- At the exact same time, Bob wants to withdraw 20. *He reads the balance(100), subtracts 20, and is about to write 80 back.*

If Alice writes her 150 *first, and then Bob writes his 80*, the final balance is 80. *If Bob writes his 80 first, and then Alice writes her 150, the final balance is 150.*

In a correct scenario, the balance should be $100 + 50 - 20 = 130$. The unpredictable interleaving of operations leads to an incorrect result. This is a race condition.

In the Philosophers project, a race condition could occur if two philosophers try to pick up the same fork simultaneously, or if multiple threads try to update a shared counter (like `meals_eaten`) without proper synchronization.

4.2. Deadlocks

A **deadlock** is a situation where two or more threads are blocked indefinitely, waiting for each other to release the resources that they need. No thread can proceed, and the entire system comes to a halt.

Analogy: Consider two cars approaching a four-way intersection with no traffic lights. Each car wants to go straight, but they both need to occupy the center of the intersection. If both cars enter the intersection simultaneously and block each other's path, they are in a deadlock. Car A is waiting for Car B to move, and Car B is waiting for Car A to move. Neither can proceed.

In the Dining Philosophers problem, a classic deadlock scenario occurs if all philosophers simultaneously pick up their *left* fork. Now, each philosopher holds one fork and is waiting for their *right* fork, which is held by their neighbor. Since everyone is waiting for a resource held by someone else, and no one can release their current resource until they get the one they're waiting for, the system deadlocks. No one can eat, and eventually, everyone starves.

4.3. Mutexes: The Solution to Race Conditions

A **mutex** (short for **Mutual Exclusion**) is a synchronization primitive that provides exclusive access to a shared resource. It's like a lock that only one thread can hold at a time. When a thread wants to access a shared resource, it first tries to acquire the mutex (lock it). If the mutex is already locked by another thread, the requesting thread waits until the mutex is released (unlocked).

Analogy: Going back to the bank account example, imagine there's a single key to the ledger. Before Alice or Bob can read or write the balance, they must first acquire the key. If Alice has the key, Bob has to wait. Once Alice is done, she releases the key, and then Bob can acquire it. This ensures that only one person is modifying the ledger at any given time, preventing incorrect balances.

In the Philosophers project, each fork is protected by a mutex. When a philosopher wants to pick up a fork, they must `pthread_mutex_lock` the corresponding mutex. When they put the fork down, they `pthread_mutex_unlock` it. This guarantees that only one philosopher can hold a specific fork at any given moment, preventing race conditions on the forks.

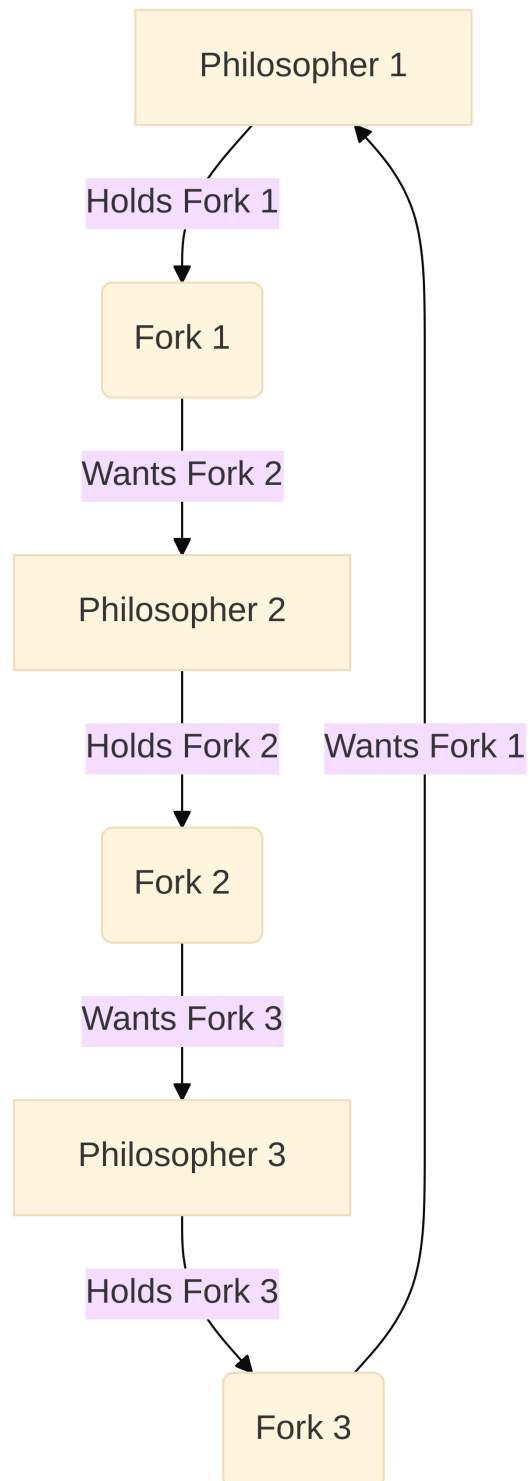
4.4. Preventing Deadlocks with Mutexes

While mutexes prevent race conditions, they can also *cause* deadlocks if not used carefully. To prevent deadlocks in the Dining Philosophers problem, common strategies include:

1. **Ordered Resource Hierarchy:** Establish a strict order for acquiring resources. For example, always pick up the lower-numbered fork first, then the higher-numbered fork. This breaks the circular wait condition.
2. **Chandy-Misra-Haas Algorithm:** A more complex distributed algorithm.
3. **Arbitrator/Monitor:** Introduce a central authority (like the `monitor_simulation` function in your project) that grants permission to eat only when both forks are available, or when it's safe to do so.
4. **Asymmetric Solution:** One philosopher picks up their right fork first, then their left, while all others pick up their left fork first, then their right. This also breaks the circular wait.

Your implementation will likely use a combination of careful fork acquisition order and a monitoring thread to detect and prevent starvation, which implicitly helps manage potential deadlocks.

Visual Analogy for Deadlock:



This diagram illustrates a circular dependency, a common cause of deadlock. Each philosopher holds one fork and waits for the other, creating a cycle where no one can proceed.

5. One-Day Completion Strategy

This project can be challenging, but with a focused approach, it's entirely possible to complete the mandatory part in a single day. Here's a suggested time-blocked strategy:

Hour 1: Setup and Argument Parsing

- **Objective:** Get your project structure in place, set up the `Makefile`, and handle command-line argument parsing.
- **Tasks:**
 - Create the `philos/` directory and the initial `Makefile` (you can use the template provided in Section 3.5).
 - Create `philos.h`, `main.c`, and `utils.c`.
 - Implement `ft_atoi` (or use your `libft` if allowed and integrated).
 - Implement `init_data` in `main.c` to parse and validate all command-line arguments (`number_of_philosophers`, `time_to_die`, `time_to_eat`, `time_to_sleep`, and optional `number_of_times_each_philosopher_must_eat`). Handle error cases for invalid input.
 - Initialize the `t_data` structure with these values.
- **Key:** Focus on robust argument parsing and basic data structure setup. Don't worry about threads yet.

Hour 2-3: Philosopher Data Structures and Mutex Initialization

- **Objective:** Define the philosopher data structure and initialize all necessary mutexes.
- **Tasks:**

- Refine `philos.h` with the `t_philo` and `t_data` structures, including `pthread_mutex_t` for forks and a `print_mutex`.
- Implement `init_philosophers` in `main.c` to allocate and initialize an array of `t_philo` structures. Assign IDs, fork IDs (left and right), and link them to the shared `t_data`.
- Initialize all fork mutexes using `pthread_mutex_init` in `init_data` or a dedicated function. Also, initialize the `print_mutex`.
- Implement `cleanup` to destroy mutexes and free allocated memory.
- **Key:** Ensure all data is correctly allocated and linked, and all mutexes are properly initialized. This lays the groundwork for threading.

Hour 4-6: Thread Creation and Basic Philosopher Routine

- **Objective:** Create philosopher threads and implement their basic lifecycle (thinking, eating, sleeping) without complex synchronization.
- **Tasks:**
 - Implement `get_time_ms` and `ft_usleep` in `utils.c`.
 - Implement `print_status` in `utils.c` (remember to use the `print_mutex`!).
 - Implement `philosopher_routine` in `main.c`. For now, have it just print its ID, then `usleep` for a bit, and loop. Don't implement fork logic yet.
 - In `main`, use `pthread_create` to launch all philosopher threads. Set `start_time` in `t_data` before creating threads.
 - Use `pthread_join` for each thread in `main` after the simulation logic (or `pthread_detach` if you prefer, but `join` is safer for debugging initially).
- **Key:** Get threads running and logging their basic states. Verify that all philosophers start and print messages.

Hour 7-9: Implementing Fork Logic and Synchronization

- **Objective:** Integrate fork acquisition and release using mutexes, and implement the eating, sleeping, and thinking phases correctly.
- **Tasks:**

- Implement `take_forks` and `put_down_forks` in `main.c` using `pthread_mutex_lock` and `pthread_mutex_unlock`.
- Modify `philosopher_routine` to call `take_forks`, `philo_eat`, `put_down_forks`, `philo_sleep`, and `philo_think` in the correct sequence.
- Implement `philo_eat`, `philo_sleep`, and `philo_think` functions, including `usleep` for the specified times and updating `last_meal_time` and `meals_eaten`.
- **Crucial:** Address the deadlock problem. A common approach is to make philosophers pick up forks in a specific order (e.g., left then right, or odd/even ID strategy).
- **Key:** This is where the core logic comes together. Test frequently with different numbers of philosophers to observe behavior.

Hour 10-12: Monitoring and Death Condition

- **Objective:** Implement the monitoring logic to detect philosopher death and handle the optional meal count limit.
- **Tasks:**
 - Implement `monitor_simulation` in `main.c`. This function will run in the main thread (or a separate monitor thread).
 - Inside `monitor_simulation`, continuously check each philosopher's `last_meal_time` against `time_to_die`. If a philosopher starves, set `simulation_should_stop` to 1 and print the

death message. * If `number_of_times_each_philosopher_must_eat` is provided, check if all philosophers have eaten at least that many times. If so, set `simulation_should_stop` to 1. * Ensure `monitor_simulation` correctly handles the `simulation_should_stop` flag to gracefully terminate philosopher threads. * Make sure your `print_status` function is thread-safe and doesn't print messages after a philosopher has died if the simulation is stopping. * **Key:** Thoroughly test the death condition and the meal count termination. This is often where subtle bugs related to timing and synchronization appear.

Final Review and Testing

- **Objective:** Ensure your code is clean, adheres to Norminette, and passes all test cases.
- **Tasks:**
 - Run `norminette` on all your files and fix any errors.
 - Test with various argument combinations, including edge cases (e.g., 1 philosopher, very short `time_to_die`).
 - Look for memory leaks using `valgrind`.
 - Ensure your output messages are exactly as specified in the subject.

6. Pro Tips to Avoid Common Evaluation Failures

Passing the Philosophers project requires not just functional code but also adherence to strict rules and best practices. Here are some pro tips to help you avoid common pitfalls during evaluation:

1. **Norminette is King:** Seriously, run `norminette` constantly. A single Norminette error can result in a 0. Get into the habit of checking it after every few lines of code.
2. **No Global Variables (Mandatory Part):** The subject explicitly forbids global variables for the mandatory part. This is a common mistake. Pass all necessary data through function arguments or structures.
3. **Thread Safety for Shared Data:** Any data that can be accessed or modified by multiple threads *must* be protected by a mutex. This includes `last_meal_time`, `meals_eaten`, and the `simulation_should_stop` flag. The `print_mutex` is also crucial for preventing garbled output.
4. **Precise Timing:** Use `gettimeofday` for accurate timestamps. Your `time_to_die` check must be precise. The subject states: "A message announcing a philosopher's death must be displayed within 10 ms of their actual death." This is a strict requirement.
5. **usleep vs. sleep:** Always use `usleep` for microsecond precision. `sleep` (seconds) is too coarse for this project. Implement your own `ft_usleep` that can

handle small delays and potentially check for `simulation_should_stop` to allow for quicker termination.

6. **Deadlock Prevention:** The most common deadlock scenario is when all philosophers pick up their left fork simultaneously and then wait indefinitely for their right fork. Implement a strategy to prevent this. Common solutions include:
 - **Odd/Even Philosopher Strategy:** Philosophers with odd IDs pick up their left fork then right. Philosophers with even IDs pick up their right fork then left. This breaks the symmetry.
 - **Arbitrator/Monitor:** A central entity (your `monitor_simulation` or a dedicated mutex) controls access to forks, ensuring that a philosopher can only pick up both forks if they are available.
7. **Starvation Prevention:** While preventing deadlock, ensure no philosopher starves indefinitely. Your `time_to_die` mechanism should handle this. The monitoring thread is key here.
8. **Memory Leaks:** Use `valgrind` regularly (`valgrind ./philo [args]`) to check for memory leaks. All `malloc` ed memory must be `free` d. This includes your `t_data` structure's internal allocations (fork mutexes array, philosophers array).
9. **Error Handling:** Check return values of `pthread_create` , `pthread_mutex_init` , `malloc` , etc. Handle errors gracefully (e.g., print an error message and exit, or clean up resources).
10. **Output Format:** The logging format is strict: `timestamp_in_ms X has taken a fork` , `timestamp_in_ms X is eating` , etc. Ensure your `print_status` function adheres to this exactly.
11. **Race Conditions on `simulation_should_stop` :** The `simulation_should_stop` flag (or similar termination flag) must also be protected by a mutex if multiple threads can modify it, or if its value is read by multiple threads and needs to be consistent. However, a common pattern is for the monitor thread to set it, and philosopher threads to only read it, which can sometimes avoid a dedicated mutex if the flag is a `volatile` atomic type or carefully managed.
12. **Philosopher ID vs. Array Index:** Be careful with 0-based array indexing vs. 1-based philosopher IDs for printing. Philosopher 1 is at index 0, Philosopher N is at index N-1.
13. **Testing Edge Cases:** Test with:

- `./philos 1 800 200 200` : One philosopher should die quickly as they can't pick up two forks.
- `./philos 5 800 200 200` : Standard case.
- `./philos 5 800 200 200 7` : With meal limit.
- Very large numbers of philosophers (e.g., 200) to stress test your synchronization.

By following these tips and the provided skeleton, you'll be well on your way to successfully completing the 42 Philosophers project.